Advanced Topics in Automation and Control Engineering

Aerial Robotics Laboratory

telekyb3

Dr Marco Tognon
Dr Gianluca Corsini

RAINBOW Team

Inria cnrs 50 IRISA Université de Rennes

telekyb3

What?

- Open-source **collection** of **software** (and **hardware**) for **multi-rotor** unmanned **aerial vehicles**

When and **where?**

- Around **2015** at **LAAS** (> 10y ago!)
- Author is Anthony Mallet (IR CNRS)

Who?

1. LAAS - CNRS (FR)
2. IRISA - Inria (FR)
3. University of Twente (NL)
4. University College London (UK)

Statistics

Growing community among several partners

Institution	LAAS-CNRS (FR)	IRISA-Inria (FR)	University of Twente (NL)	University College of London (UK)
Users	≥ 5	≥ 10	≥ 5	1?
Maintainers	2 (SW)	1 (SW/HW)	1 (HW)	–

Why telekyb3?

1. **Modularity, Reusability and Interchangeability**
   - Several components: each one implementing one (or more) functionality(ies)
   - **Interface-based** design: components use interfaces to communicate
2. **Formal description language**
   - Allows software validation and verification, and well-defined behavior
3. **Middleware abstraction**
   - Component description is unaware of middleware implementation (*templates*)
4. **Low-level access**
   - Full control on any low-level component (either hardware or software)
5. **Variety** of **aerial robots**
   - **Single** and **multi-robot** systems with **different rotor configurations**
6. **Variety** of **applications**

The 3 “pillars” of telekyb3

1. Part of the git.openrobots.org project
   - *“Robotics Open Source Software developed at LAAS - CNRS”*
2. Delivered by means of robotpkg
   - Compilation from source on host CPU
   - Provides a PPA and binary packages for debian-based systems (.deb)
   - Stable versions of the software and regular releases
3. Heavily based on GenoM3
   - “[...] a tool to design real-time software architectures”

GenoM3

- Tool designed to write **independent** and **reusable components**
  - **Component** = “[...] a ***server*** that provides a number of services and communicates through data ports with other components in the system”
  - **Component description files** (data types, services, ports)
- Provides **middleware abstraction**
  - Templates to select target middleware (e.g., pocoLibs, ROS, Yarp, (soon) ROS2, …)
- Source code automatically generated
  - Target middleware
  - Main component routines
- Allow interfacing with **external *clients*** (user applications)
  - **Genomix**: middleware-independent interface
  - Support for different scripting programming languages: TCL, Python, MATLAB/Simulink

Main GenoM3 components in telekyb3

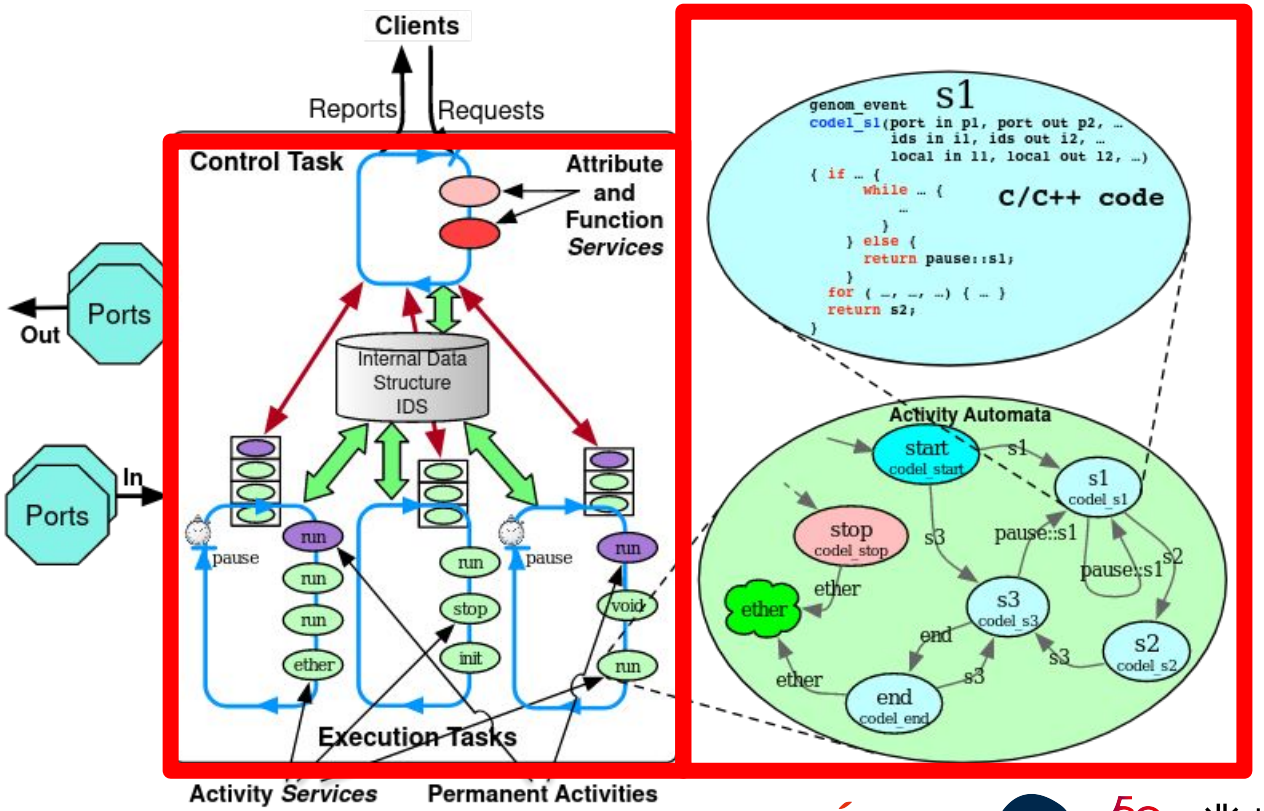
- **Control**:
  - *nhfc-genom3*: cascade PID for under-actuated aerial vehicles
  - *uavpos/uavatt-genom3*: positional and attitude controllers for fully-actuated aerial vehicles
  - *phynt-genom3*: admittance filter + wrench observer
- **Estimation**:
  - *pom-genom3*: an Uscented Kalman Filtering for robot state estimation fusing together several sensors
- **Motion**:
  - *maneuver-genom3*: kinematic trajectory generator
- **Robot interfaces**:
  - *rotorcraft-genom3*: interface with robot low-level hardware
- **Sensors**:
  - *optitrack-genom3*: interface for OptiTrack Motion Capture System
  - *dynamixel-genom3*: interface for Dynamixel (servo) motors

Documentation for <COMPONENT_NAME>-genom3 at:
https://git.openrobots.org/projects/<COMPONENT_NAME>-genom3/pages/README

Inside a GenoM3 component

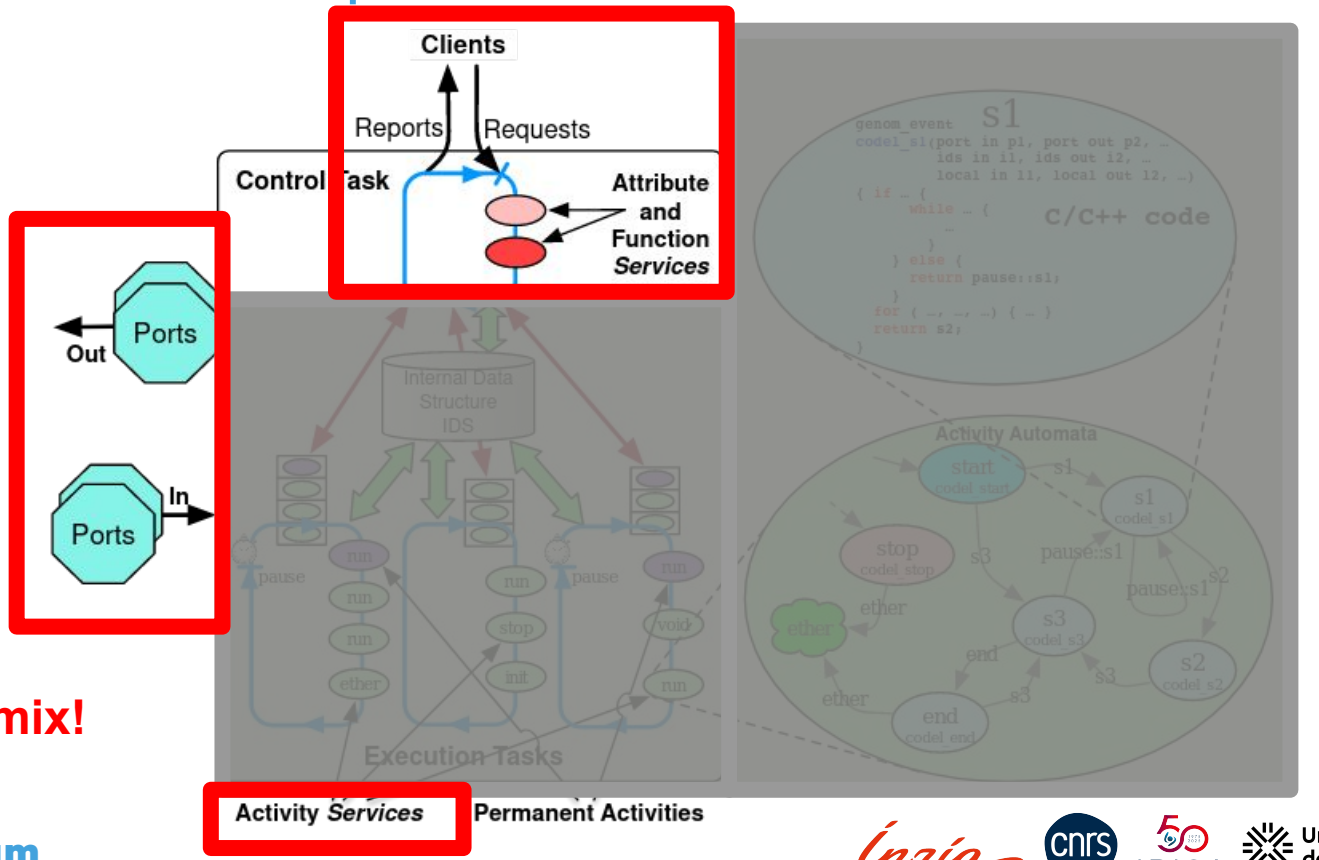
for developers!

1. IDS
2. Ports
3. Tasks
4. Services
5. Codels



Inside a GenoM3 component

1. IDS
2. Ports
3. Tasks
4. Services
5. Codels



for users!

using genomix!

Let’s use an example: rotorcraft-genom3

https://git.openrobots.org/projects/rotorcraft-genom3/pages/README

A genomix client can

- *write data to* input (in) ports
- *read data from* output (out) ports

Let's use an example: rotorcraft-genom3

https://git.openrobots.org/projects/rotorcraft-genom3/pages/README

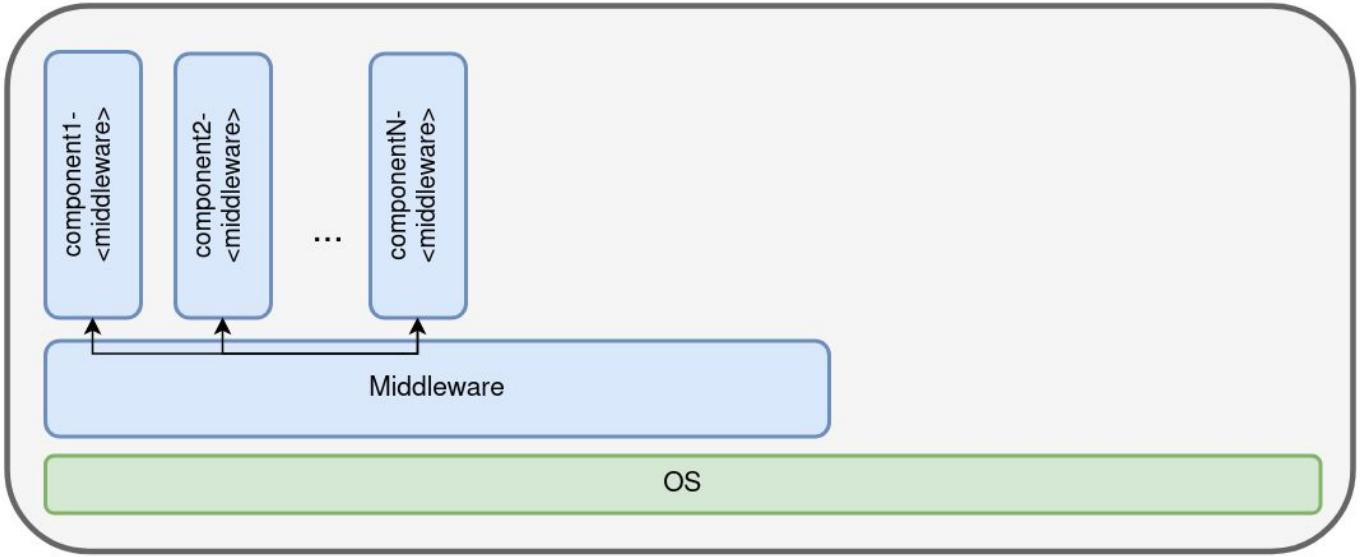
Services

- get_sensor_rate (attribute)
- set_sensor_rate (function)
- get_battery (attribute)
- set_battery_limits (attribute)
- get_calibration_param (attribute)
- set_calibration_param (attribute)
- get_imu_calibration (attribute)
- set_imu_calibration (function)
- get_imu_filter (function)
- set_imu_filter (function)
- get_imu_temp (attribute)
- set_timeout (attribute)
- set_ramp (attribute)
- get_cmd_filter (function)
- set_cmd_filter (function)
- connect (activity)
- pconnect (activity)
- disconnect (activity)
- monitor (activity)
- disable_motor (function)
- enable_motor (function)
- set_pid (function)
- calibrate_imu (activity)
- calibrate_mag (activity)
- set_zero (activity)
- set_zero_velocity (activity)
- start (activity)
- servo (activity)
- set_velocity (function)
- set_throttle (function)
- stop (activity)
- log (activity)
- log_stop (function)
- log_info (function)
- get_sensor_average (activity)

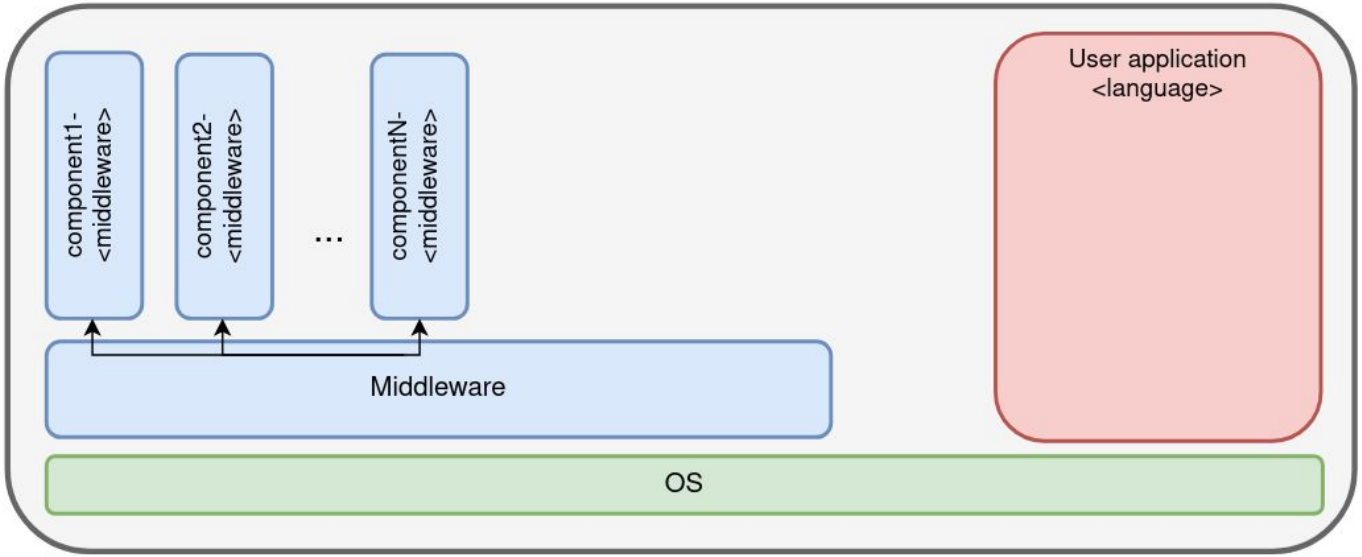
A genomix client can *call* services
3 types of services:

- ***attribute***: (usually) like *"get/set of a class in object-oriented programming"*
- ***function***: like attributes, but they run an internal function (a ***codel***) which can be more complex than just set/get
- **activities**: state machines that execute internal "functions" (1+ **codels**) for a bounded/unbounded amount of time and/or until certain conditions are met

Interactions between Genom3 components and clients?

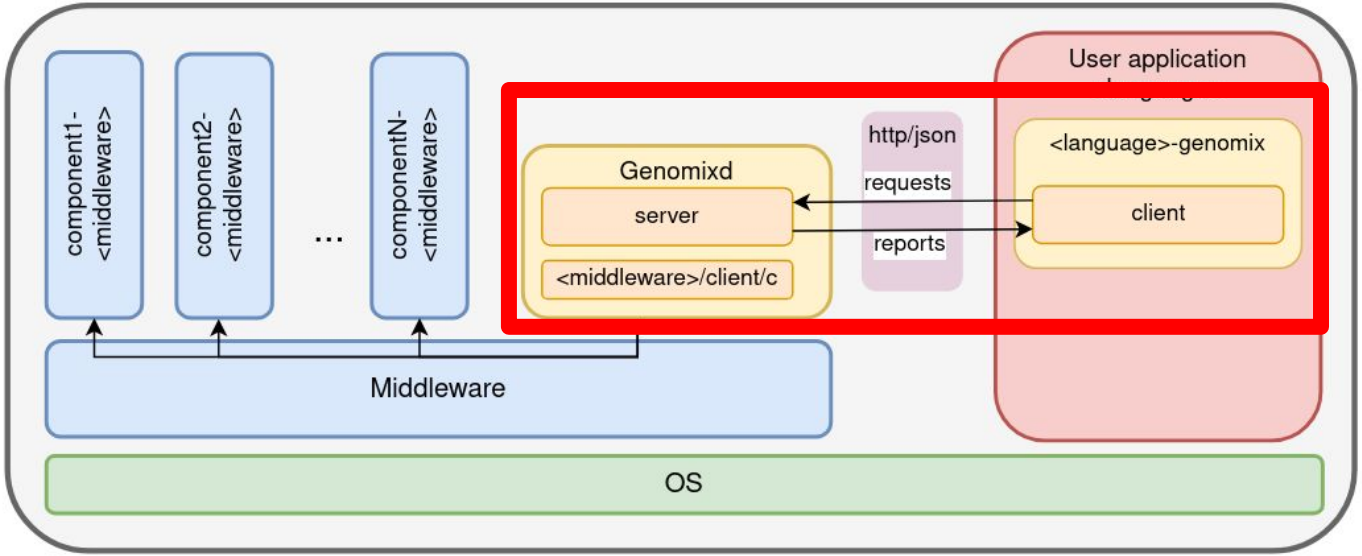


Interactions between Genom3 components and clients?



Interactions between Genom3 components and clients?

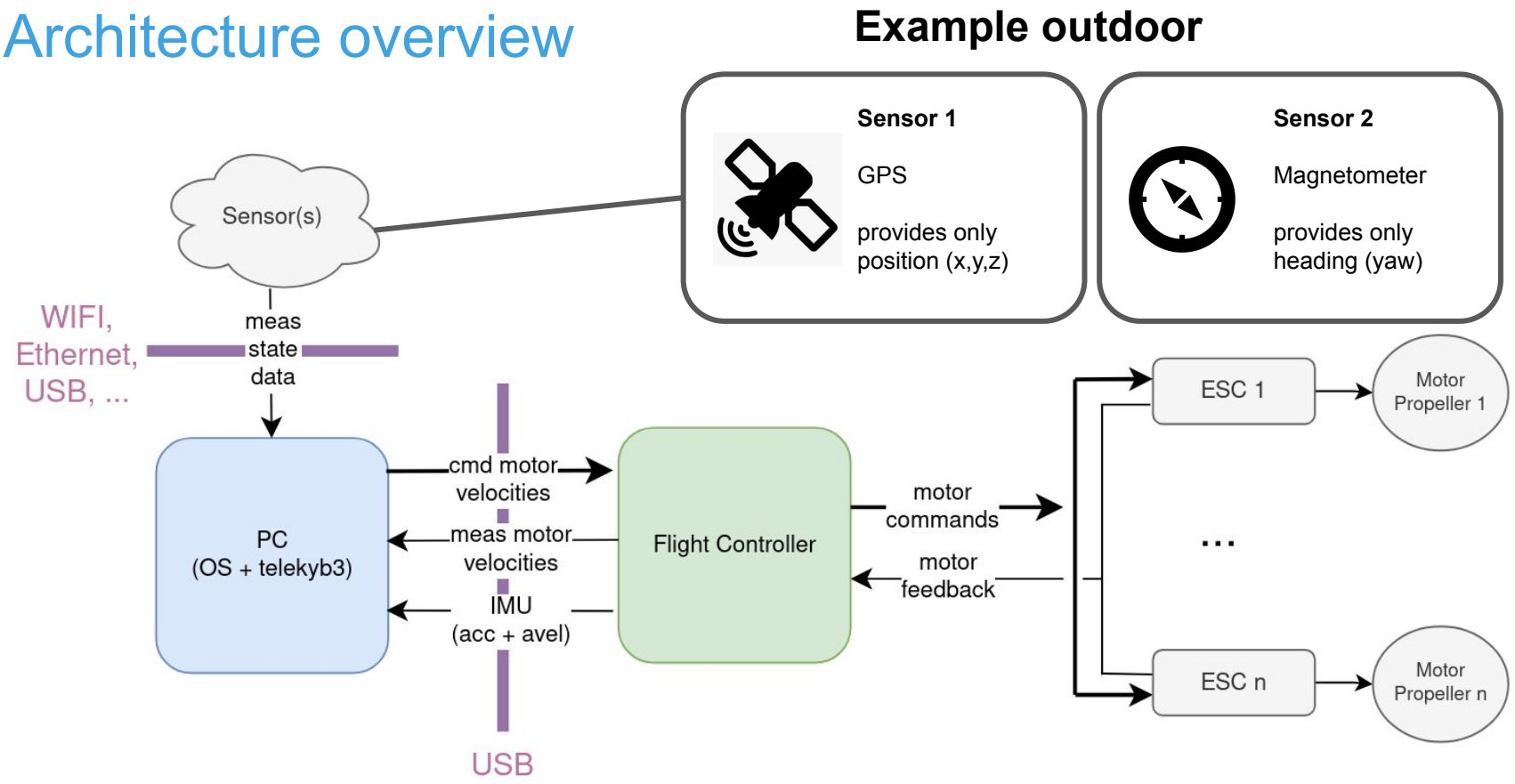
Genomix = daemon **server** (*genomixd*) + **client** (*<language>-genomix*)



where

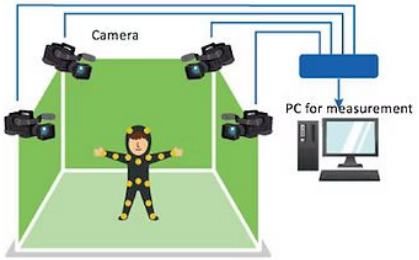
- *<middleware> = [ROS | pocoLibs | YARP …]*
- *<language> = [tcl | python | matlab]* , e.g. *python-genomix*

Architecture overview



Architecture overview

Example indoor

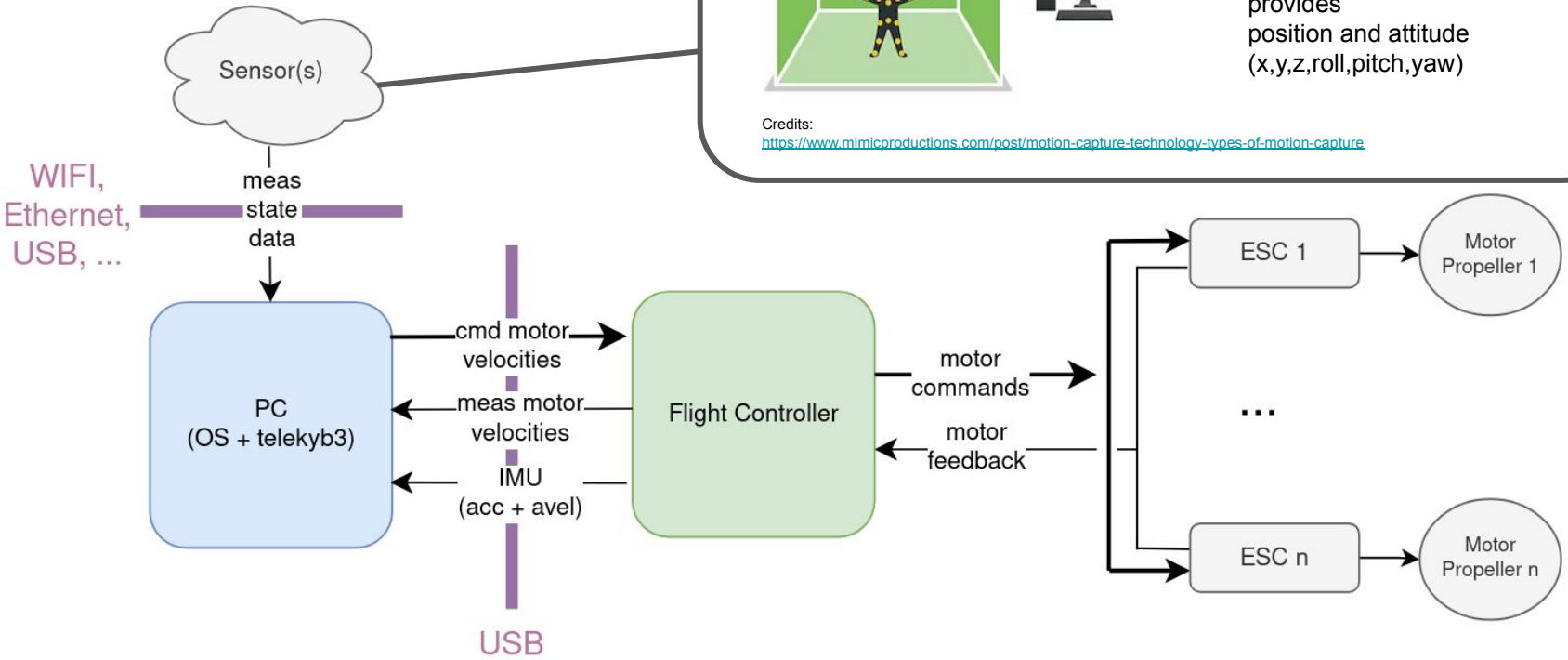


Sensor

Motion Capture System

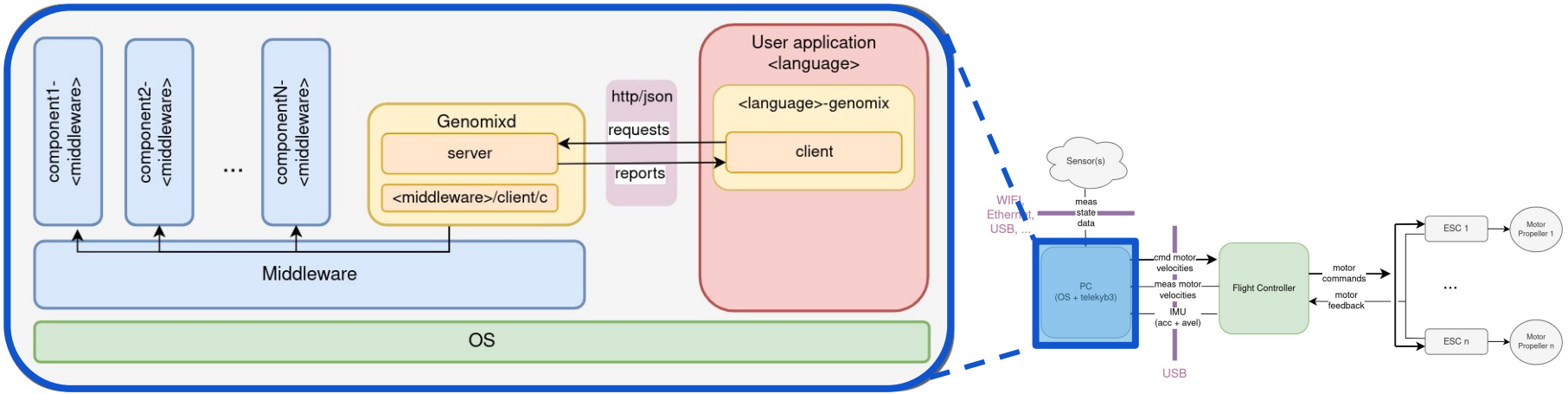
provides
position and attitude
(x,y,z,roll,pitch,yaw)

Credits:
https://www.mimicproductions.com/post/motion-capture-technology-types-of-motion-capture



What inside the PC?

The architecture of before!

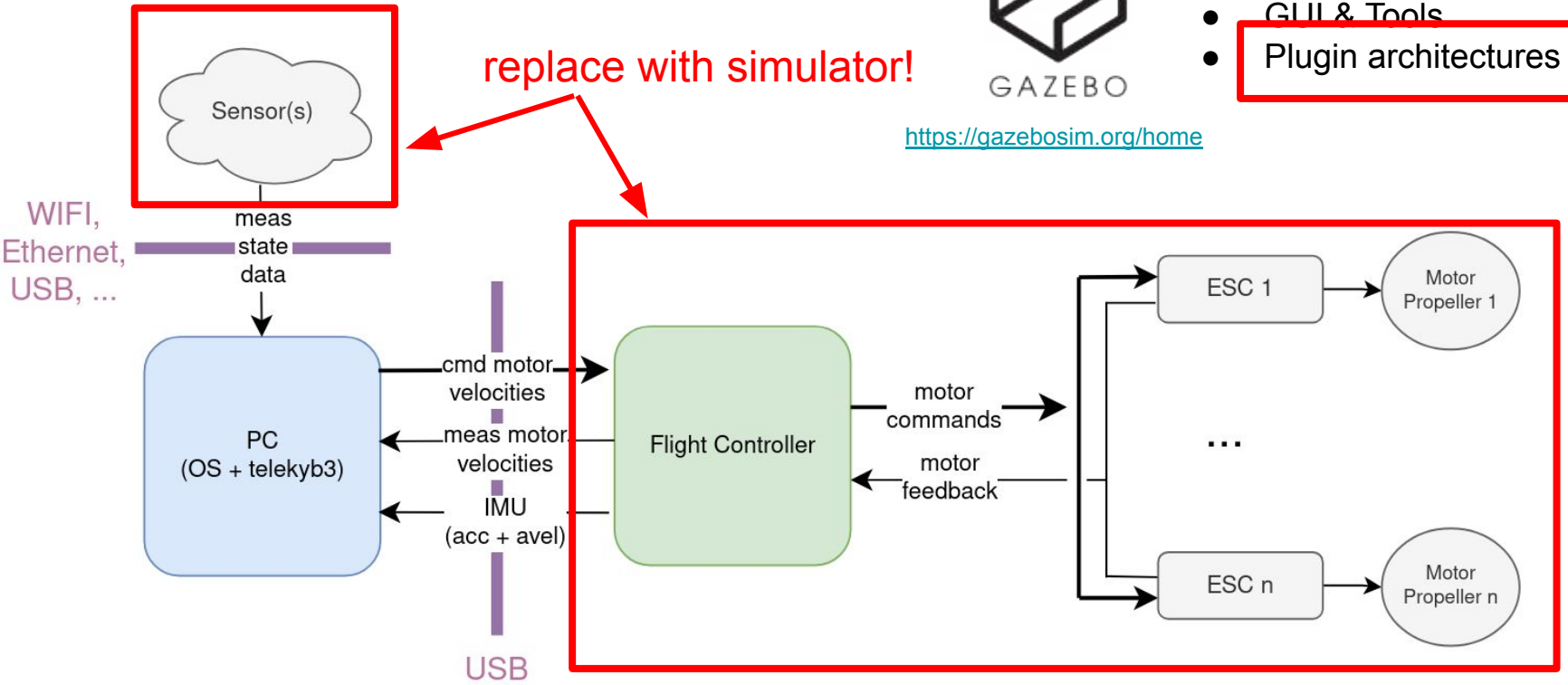


example:

- <middleware> = ROS

- <language> = Python

What about simulations?

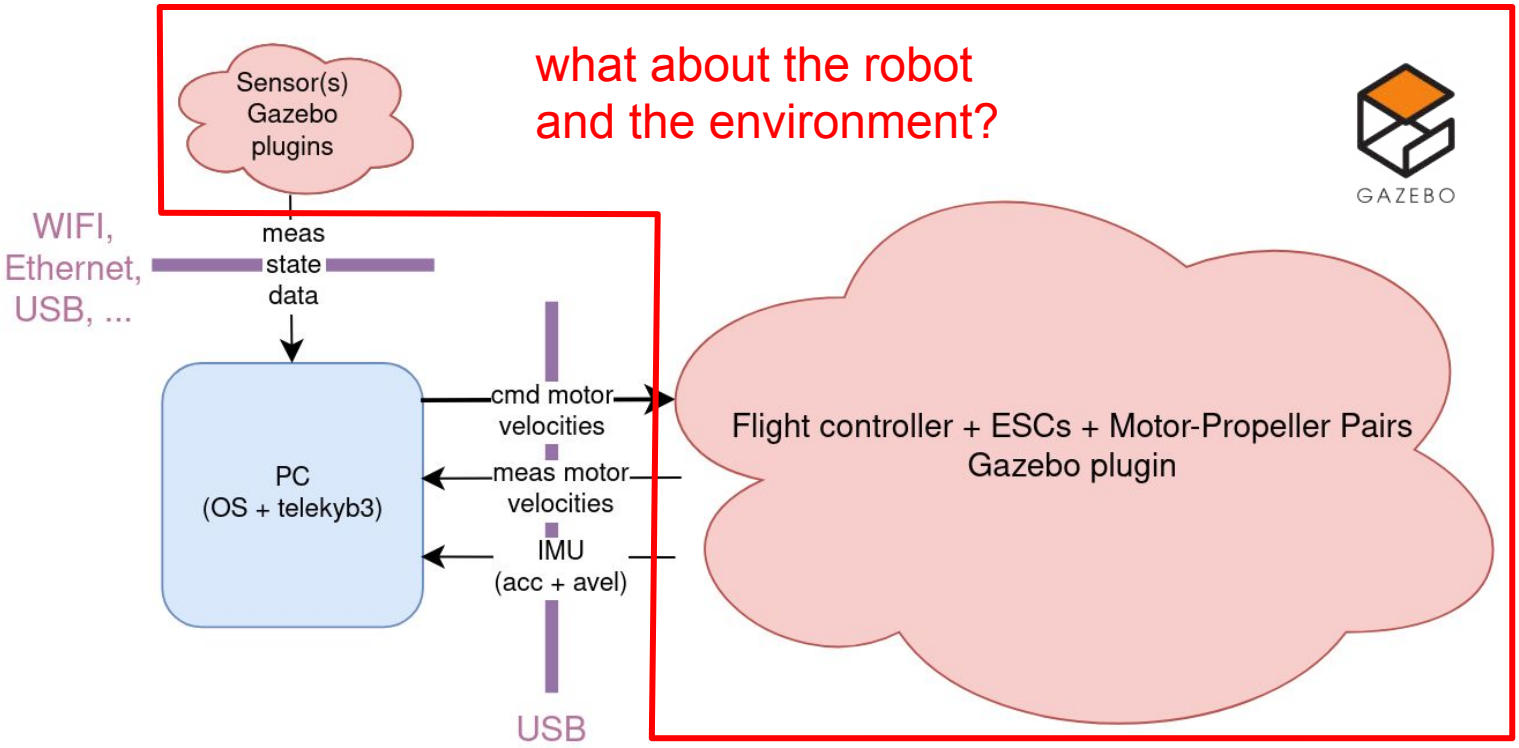


GAZEBO

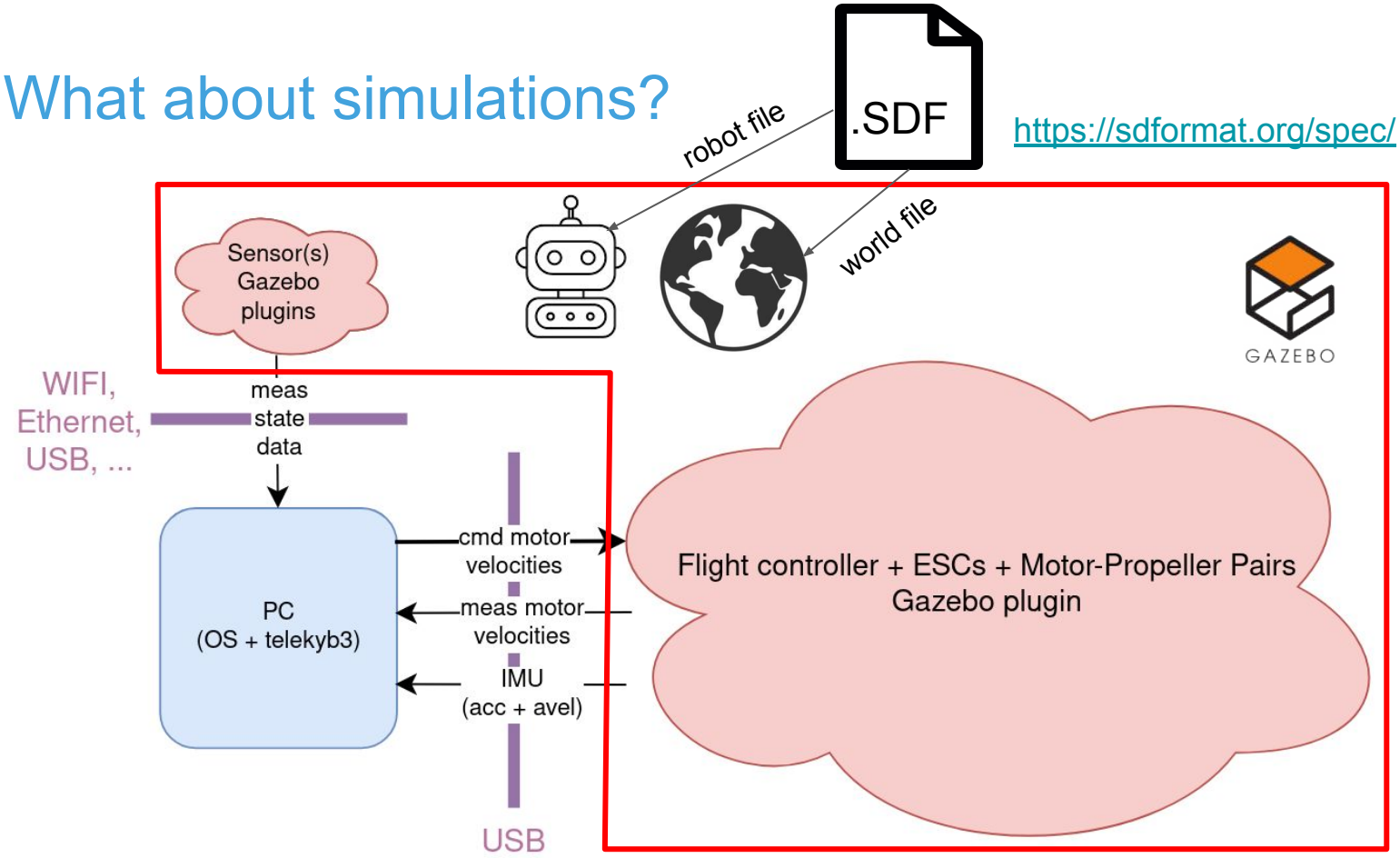
- Physics Simulation
- 3D Rendering
- GUI & Tools
- Plugin architectures

https://gazebosim.org/home

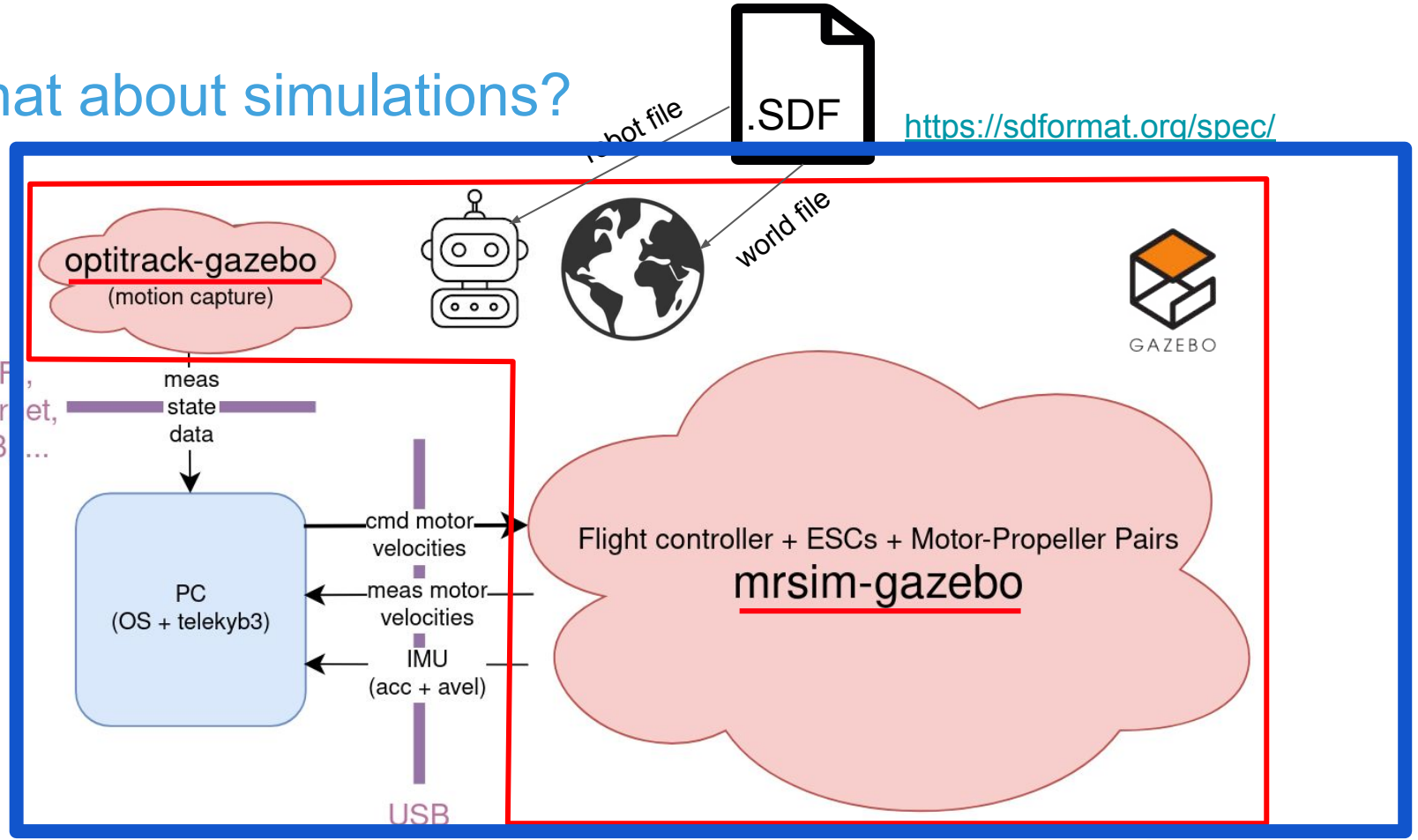
What about simulations?



What about simulations?

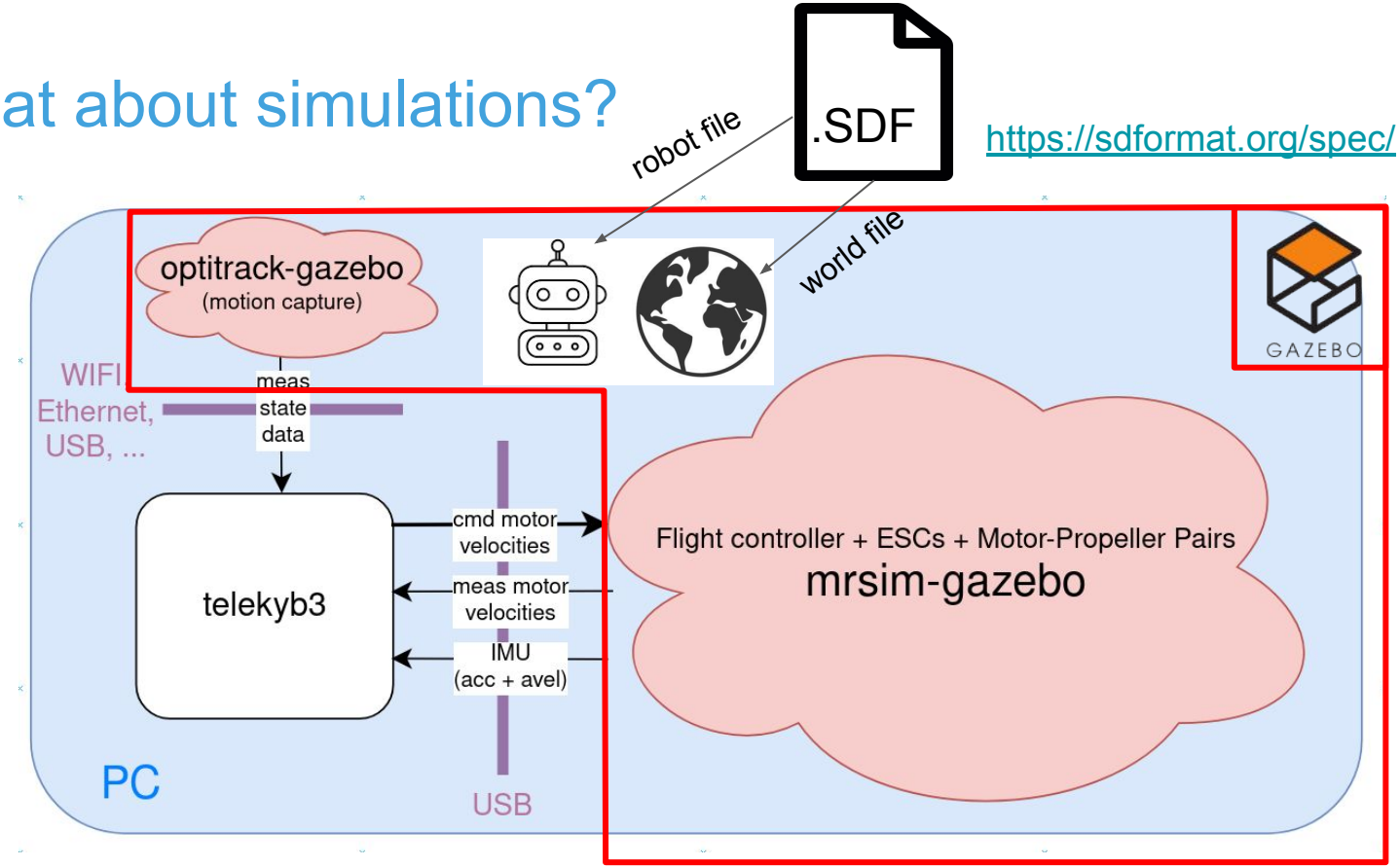


What about simulations?



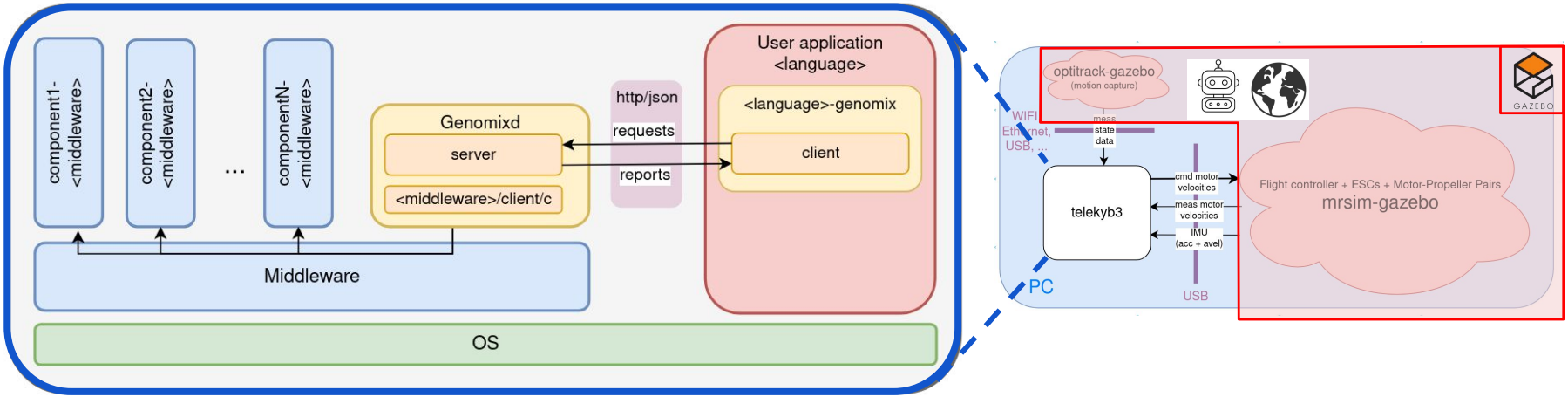
NB: All runs inside PC now!

What about simulations?



What inside the PC?

The architecture of before!



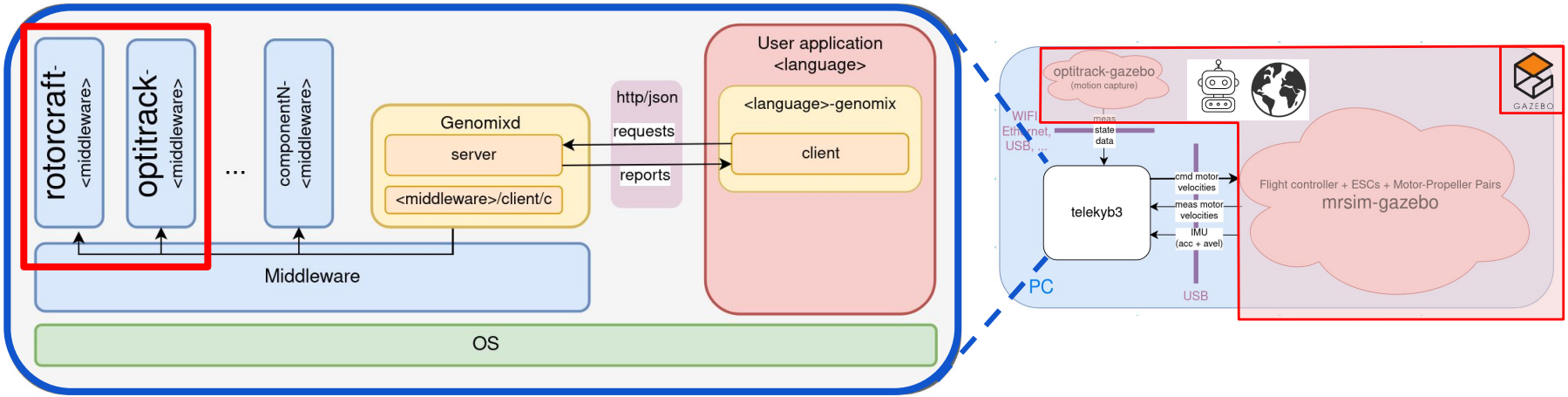
example:

- <middleware> = ROS

- <language> = Python

What inside the PC?

How the components *rotorcraft* and *optitrack* interact with the (simulated) robot and sensors?



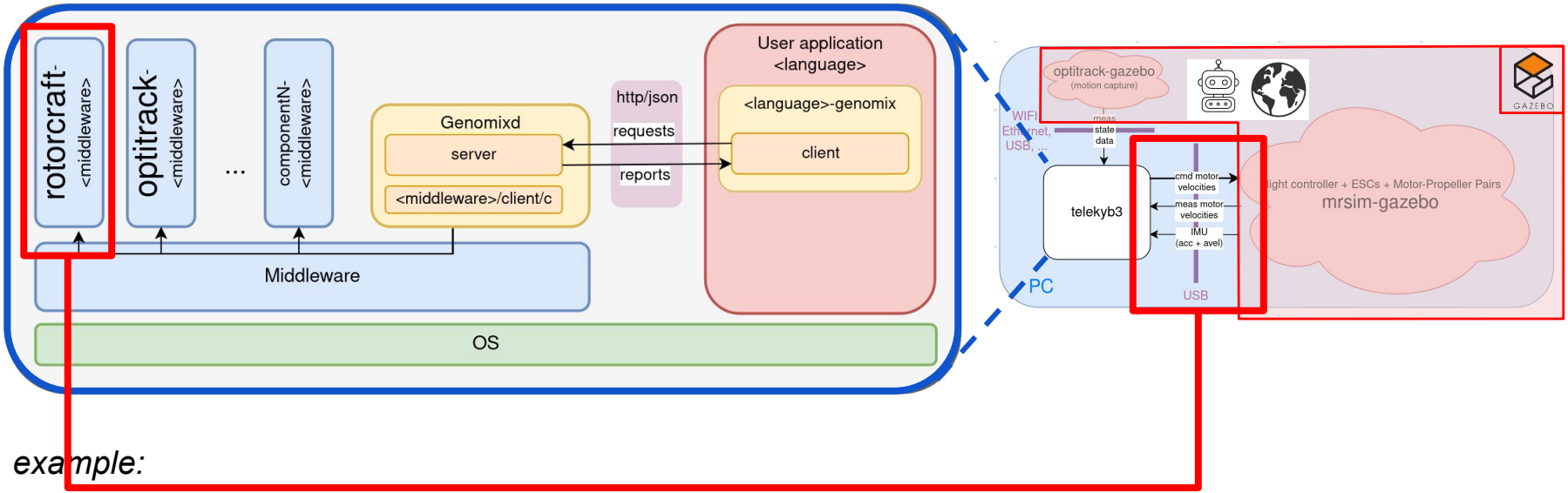
example:

- <middleware> = ROS

- <language> = Python

What inside the PC?

How the components *rotorcraft* and *optitrack* interact with the (simulated) robot and sensors?



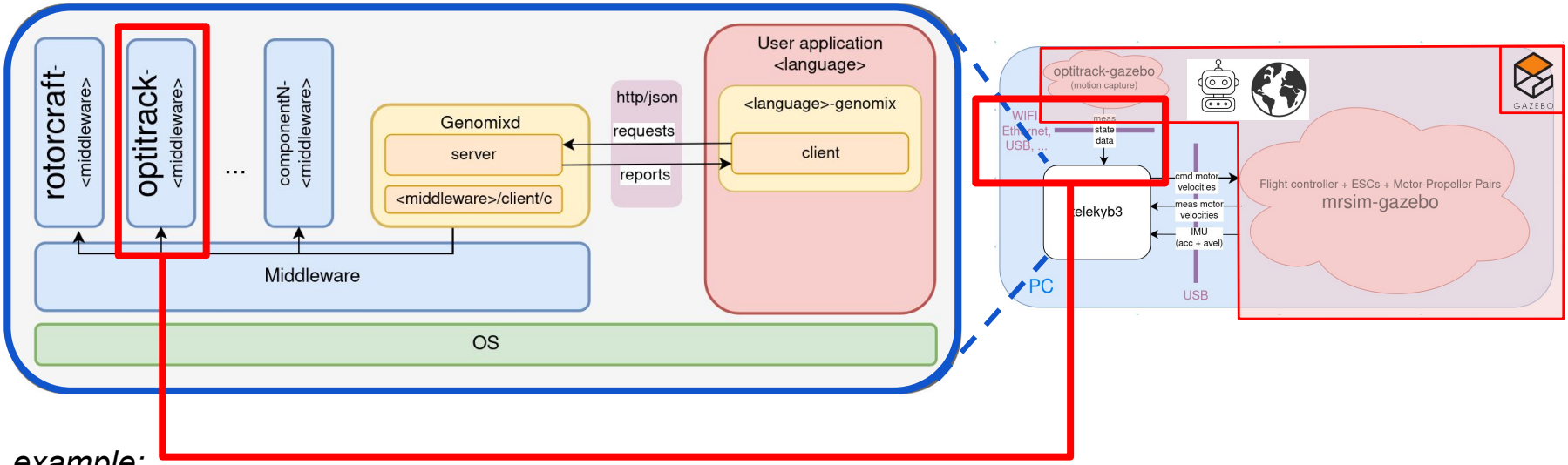
example:

- *<middleware> = ROS*
- *<language> = Python*

connects to the (simulated) USB

What inside the PC?

How the components *rotorcraft* and *optitrack* interact with the (simulated) robot and sensors?



example:

- *<middleware> = ROS*
- *<language> = Python*

connects to the (simulated) (socket) port

FYI: OptiTrack uses sockets (UDP/TCP)

What a telekyb3 users does?

1. Start PC (run OS)
2. Start middleware (ROS: roscore / pocolibs: h2 init)
3. Start the components (run processes)
4. Start the genomix server (genomixd)
5. Start the use application which contains genomix client (e.g. python-genomix)
6. Quit user application
7. Stop genomix server (genomixd)
8. Stops the components (kill processes)
9. Stop middleware
10. Shutdown PC (stop OS)

Why having an user application?

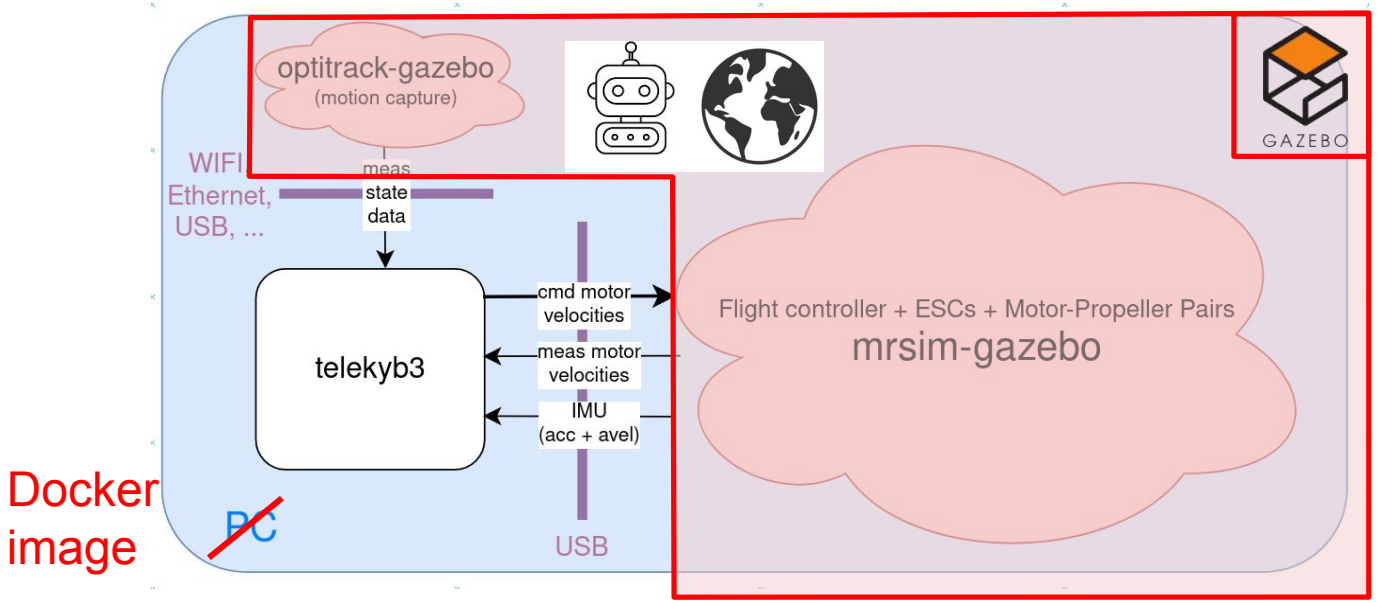
When components start, they are…

- Unconnected
  - Connect ports (call to specific service “connect_port”) between them
    - Input-output communication between components (in a “Simulink-like fashion”)
    - Allow configuration at runtime (modularity and interchangeability)
- Only initialized with default parameters
  - Call services to initialize correct parameters (e.g. robot, environment)
- In “*indle”* state (i.e., usually “doing nothing”)
  - Call services to start “doing something” / stop “doing something” (safety)

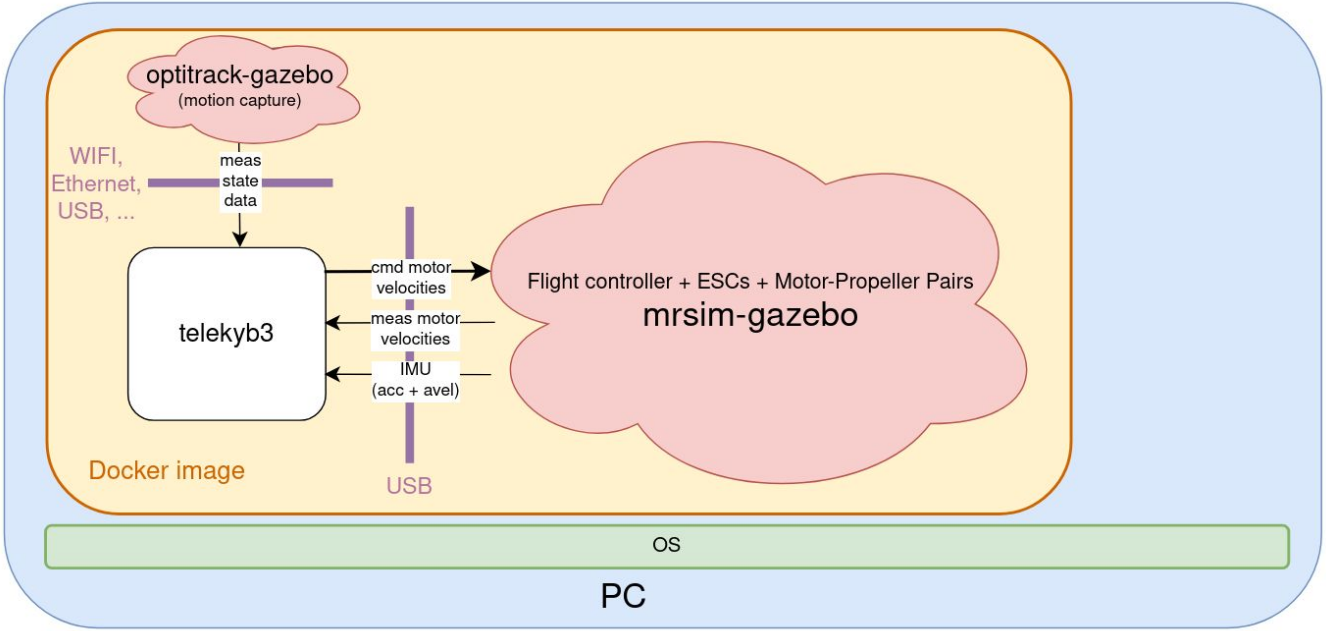
All these operations are usually left to the user application (high-level supervision)

What about Docker?

Instead of installing telekyb3 (e.g. through robotpkg) and Gazebo, everything is available inside the provided Docker image *tk3lab!*

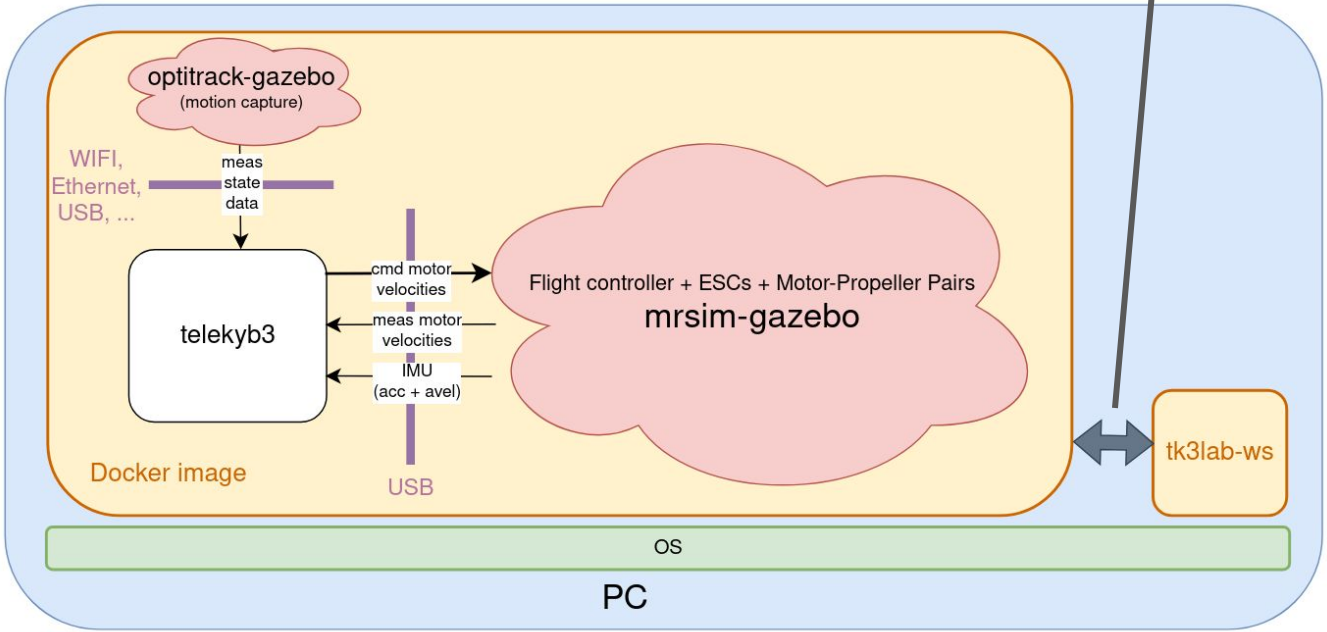


What about Docker?



What about Docker?

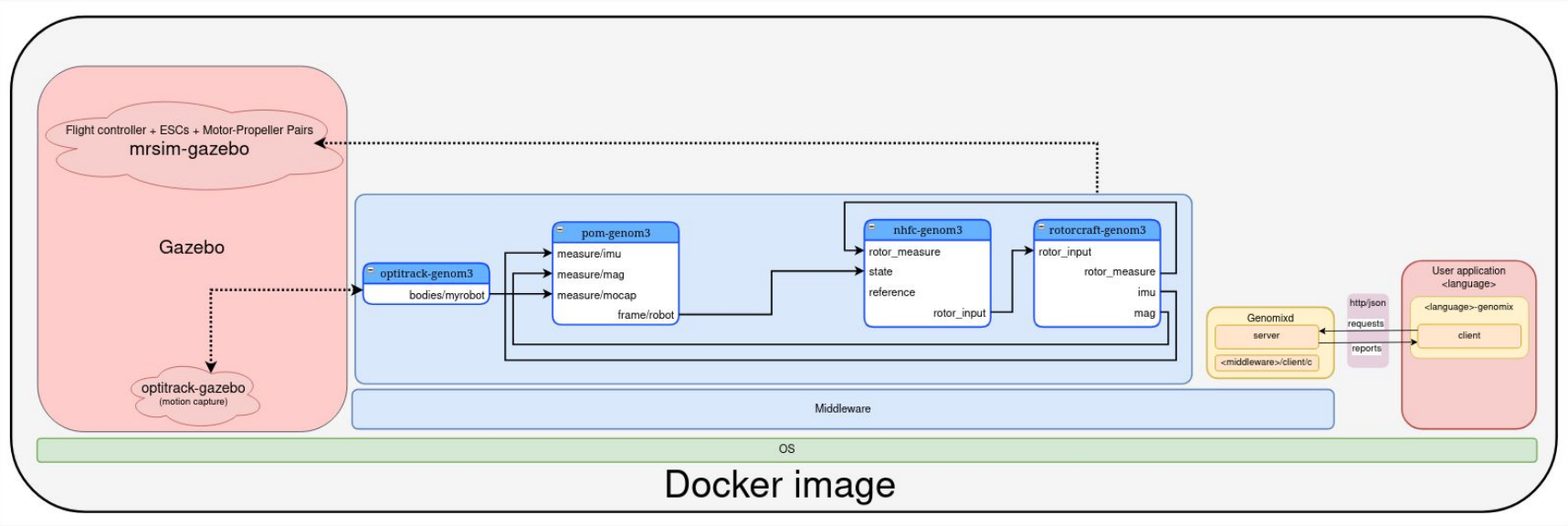
When running Docker image: $HOME/tk3lab-ws is shared workspace between host and container (Docker image)



NB: When stopping Docker image everything outside shared workspace is lost!

Example: quadrotor simulation in the *tk3lab* Docker image

Let's see in details what is happening



How to extend the framework?

1. ~~Write a new GenoM3 component~~
   a. ~~Relatively complex development~~
   b. ~~Easy to reuse (reusability)~~
   c. ~~Use and provide same interfaces (interchangeability)~~
   d. ~~Efficient (C/C++)~~

For expert users / developers!
Not part of this laboratory!

2. Write an user application using genomix interface
   a. Easy development (scripting language, e.g., Python)
   b. Reusable? → may depend on the implementation, quality and robustness of the code, …
   c. Interchangeable? → may require some hacking
   d. Efficient? →depends on the language used, imported libraries, …

What we will do in this laboratory!